Priority Queue : unordered array implementation

A priority queue is an abstract data type in which each element is associated with a priority.

Unlike an ordinary queue, where elements are removed according to FIFO (First-In, First-Out) order, a priority queue removes the component with the highest priority first.

A priority is a number associated with each component. Higher the numeric value, could mean higher the priority. Or lower the numeric value could mean higher priority.

In our examples, we will assume that higher the numeric value, higher the priority.

Let us relook at the interface of the priority queue before discussing the implementation using unordered list.

Interface of the priority queue:

The interface of a priority queue is same as that of queue. Component x would have a priority associated with it.

initialize()	Create an empty queue
deinitialize()	Make the queue empty
enqueue(x)	Insert x We may add in number of ways. This in turn would affect dequeue
dequeue()	Remove the element with the highest priority. This in turn depends how enqueue operation was done
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full, for bounded implementations

1. **Implementation of priority queue: Using an unordered array**

We have discussed the implementation of priority queue using an unordered list. Now, we shall discuss the priority queue using an **unordered array**. The queue structure is changed to have an array of nodes. The node structure will remain the same. The interface will also remain the same!

We do not have rear and front. Instead, we have n, which is the count of number of nodes in the priority queue.




```
#define MAXSIZE 10

struct queue

{

        job_t key_[MAXSIZE];

        int n_; // # of elements

};

typedef struct queue queue_t;


struct node

{

        job_t jobx_;

        struct node* next_;

};

typedef struct node node_t;


void init_queue(queue_t *ptr_queue);

void deinit_queue(queue_t *ptr_queue);

void enque(queue_t *ptr_queue, job_t *ptr_job);

job_t deque(queue_t *ptr_queue);
```

```
int is_empty(queue_t *ptr_queue);

int is_full(queue_t *ptr_queue);
```

The list is not in any order based on priority. Let us discuss each one of these operations.

Initialize:

Make the number of elements in the priority queue zero.

```
void init_queue(queue_t *ptr_queue)
{
        ptr_queue->n_ = 0;
}
```

Deinitialize :

There were no resources allocated during any of these processes dynamically.

Just change n to 0 indicating no elements in the priority queue.

```
void deinit_queue(queue_t *ptr_queue)
{
        ptr_queue->n_ = 0;
}
```

Is full :

Check whether the number of elements is equal to the MAXSIZE of the array of nodes.

```
int is_full(queue_t *ptr_queue)
{
        return ptr_queue->n_ == MAXSIZE;
}
```

Is empty:

Check whether the number of elements in the priority queue is 0.

```
int is_empty(queue_t *ptr_queue)

{

        return ptr_queue->n_ == 0;

}
```

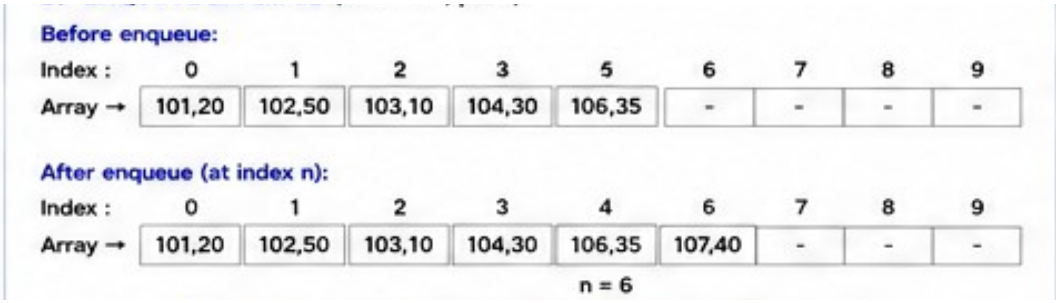
Enque :

As we have an unordered array, the enque is straight forward.

If the priority queue is full, the program is exited.

Otherwise use n as the index on the key – an array of nodes and assign the job to the array key at that location.

What if we get more than one node with the same priority. The one which came later will be added at a later position. So, while servicing the one which came earlier with the same priority will be catered to first. So, this is fine.



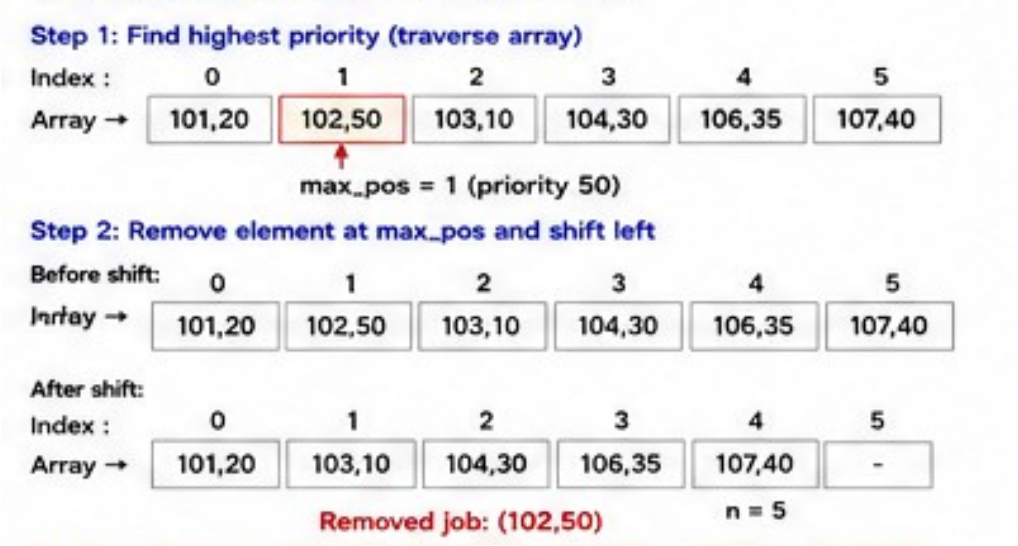
```
void enque(queue_t *ptr_queue, job_t *ptr_job)
{
        if(! is_full(ptr_queue))
        {
                ptr_queue->key_[ptr_queue->n_++] = *ptr_job;
        }
        else
        {
                printf("queue full; cannot push\n");
                exit(1);
        }
}
```

Deque:

If the queue is empty, the program is exited.

There are two parts in this algorithm.

1. Find the node of the highest priority.
   It is calling finding the biggest in an array. We track the biggest so far in an index variable called max_pos. We have to traverse the whole array as there is no order of elements.
2. Remove that node.
   Once we find the position, we copy the job in that location to a local variable.
   How to remove an element from the array?
   We move the elements left by one position starting from the position where the max priority occurs. Then we decrease the number of elements in the priority queue.




```
job_t deque(queue_t *ptr_queue)
{
        if(! is_empty(ptr_queue))
        {
                int max_pos = 0;
                int n = ptr_queue->n_;
                for(int i = 1; i < n; ++i)
                {
                        if(get_priority(&ptr_queue->key_[i]) >
                                get_priority(&ptr_queue->key_[max_pos]))
                        {
                                max_pos = i;
                        }
                }
                job_t jobx = ptr_queue->key_[max_pos];
```

```
            // remove this
            for(int i = max_pos; i < n - 1; ++i)
            {
                ptr_queue->key_[i] = ptr_queue->key_[i+1];
            }
            --ptr_queue->n_;
            return jobx;
        }
        else
        {
            printf("queue empty; cannot pop\n");
            exit(1);
        }
}
```

Efficiency of operations :

.

Enque : This requires a few operations. We add at the end without considering the priority

Deque : This requires walking through the list to find the node with the highest priority and then remove it by shifting the elements from right to left. So, the number of operations is proportional to the number of nodes in the queue.